



M



ARTIFICIAL INTELLIGENCE(Q) – JUNE 2026
KECERDASAN TIRUAN(Q) – JUNI 2026

Introduction on Natural Language Processing

Mohammad Iqbal, S.Si., M.Si., Ph. D.



OUTLINES

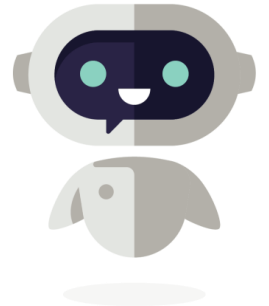


- ❑ Intro - Course
- ❑ Intro NLP
- ❑ Encoding
- ❑ Optimization Step





Natural Language Processing



What is Natural Language Processing?



A field of **designing methods and algorithms** that takes as an input or produce as output unstructured, **natural language data**.

Giving computer the ability to understand text and spoken words in much the same way human beings can.

Involves **computational linguistic**—rule-based modeling of human language and **statistics, mathematics, machine learning**.

NLP Applications



sentiment
analysis



search engine



information
retrieval

information
extraction



email
filtering



chatbot

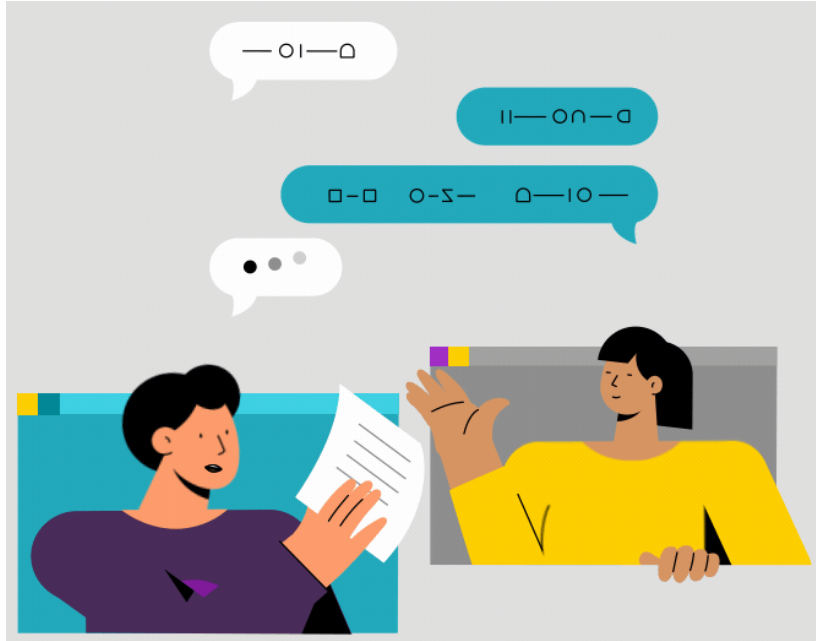


machine
translation

Human Language & Word Meaning



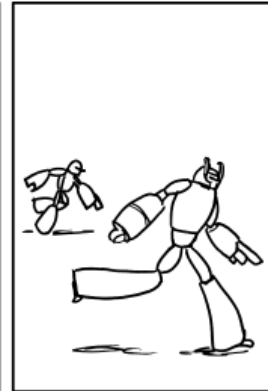
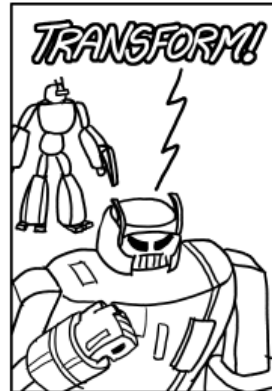
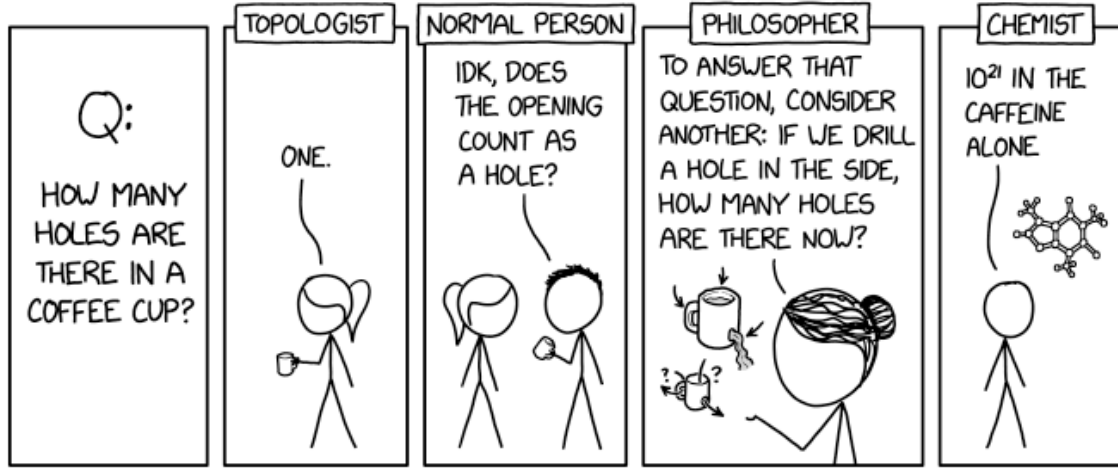
Easy to understand but hard to draw it into numeric



Complex understanding is EVEN harder

Human Language & Word Meaning

<https://xkcd.com/archive/>



Word Meaning



❑ From Oxford Language Dictionary :

- What is meant by a word, text, concept or action
- Implied or explicit significance
- Important or worthwhile quality

❑ Common linguistic way for meaning :

signifier (symbol) $\Leftrightarrow$ **signified** (idea or thing)



$\approx$ **denotational semantics**

Word Meaning in Computer...



- ❑ **WordNet** : a thesaurus containing lists of **synonym sets** and **hypernyms**.

```
import nltk  
nltk.download('wordnet')  
from nltk.corpus import wordnet as WN
```

Synonym set



```
# declare the poses
poses = {'n':'noun', 'v':'verb', 'a':'adj', 'r':'adv', 's':'adj (s)'}

# find it the synonym set (ss) of word

w = 'good'

for ss in WN.synsets(w):
    print('{:} {}'.format(poses[ss.pos()], ' '.join([l.name() for l in ss.lemmas()])))
```

```
noun: good
noun: good,goodness
noun: good,goodness
noun: commodity,trade_good,good
```

```
adj: good
adj (s): full,good
adj: good
adj (s): estimable,good,honorable,respectable
adj (s): beneficial,good
adj (s): good
adj (s): good,just,upright
adj (s): adept,expert,good,practiced,proficient,skillful,skilful
adj (s): good
```

```
adj (s): dear,good,near
adj (s): dependable,good,safe,secure
adj (s): good,right,ripe
adj (s): good,well
adj (s): effective,good,in_effect,in_force
adj (s): good
adj (s): good,serious
adj (s): good,sound
adj (s): good,salutary
adj (s): good,honest
adj (s): good,undecomposed,unspoiled,unspoil
adj (s): good
adv: well,good
adv: thoroughly,soundly,good
```

Hypernyms

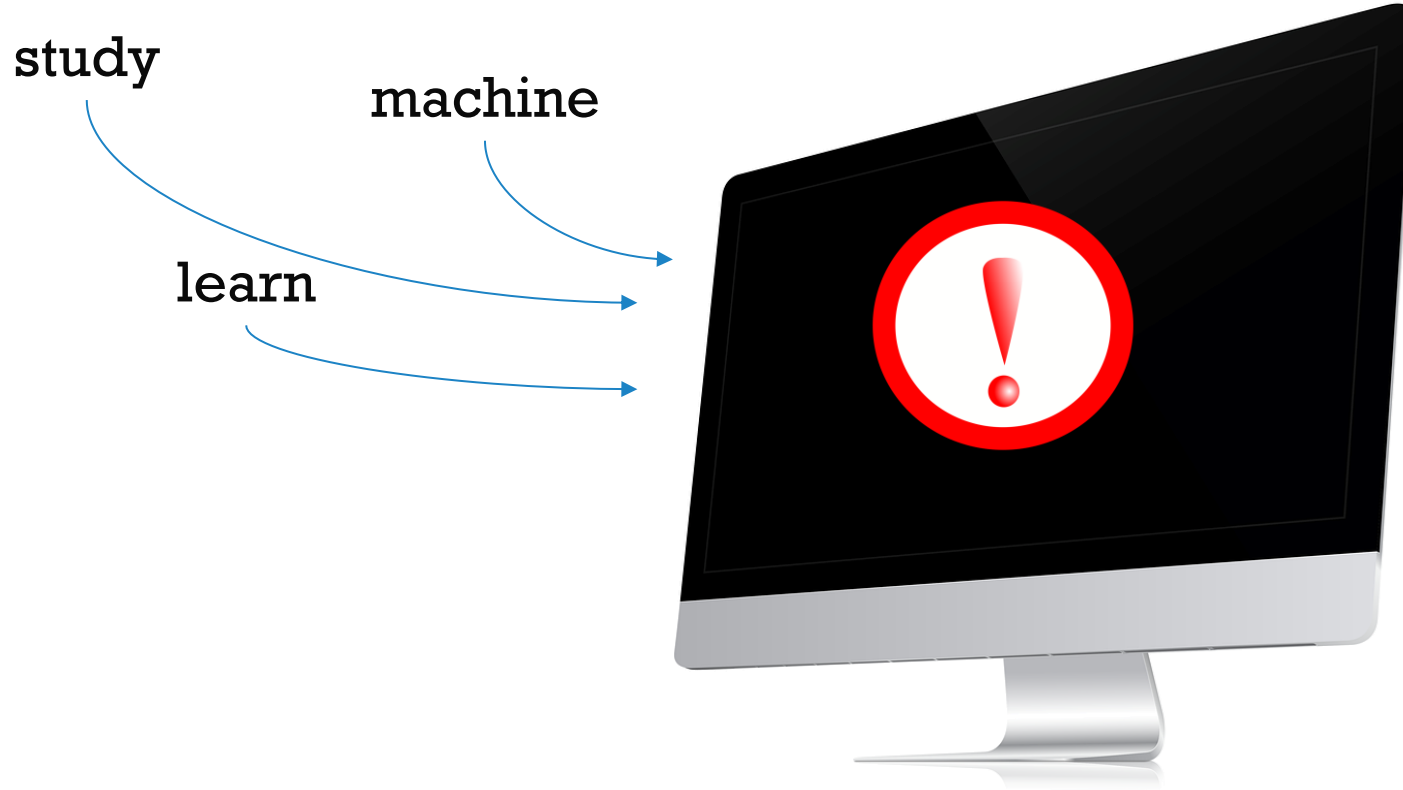


```
# hypernyms
```

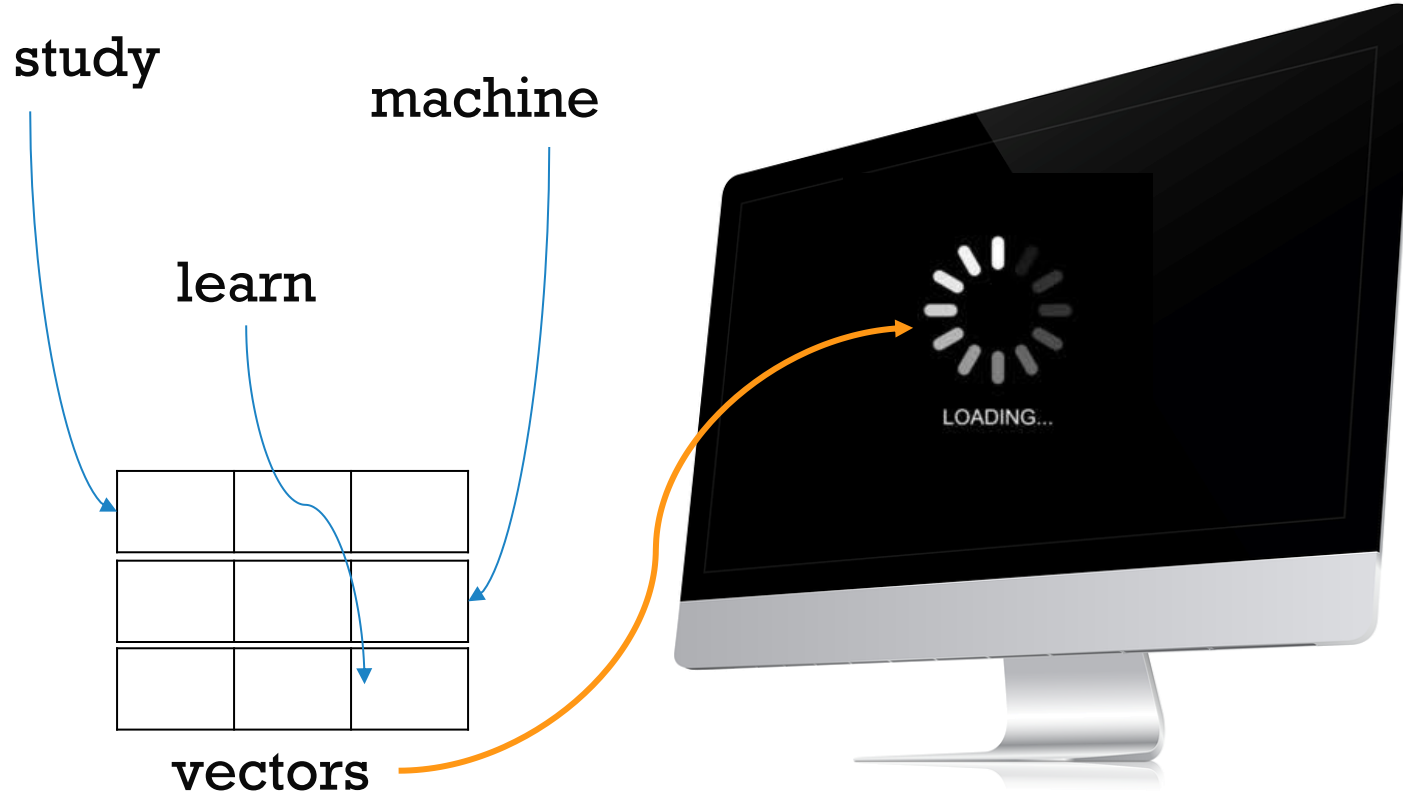
```
w = WN.synset('owl.n.01')
h = lambda s: s.hypernyms()
list(w.closure(h))

[Synset('bird_of_preyn.01'),
 Synset('bird.n.01'),
 Synset('vertebrate.n.01'),
 Synset('chordate.n.01'),
 Synset('animal.n.01'),
 Synset('organism.n.01'),
 Synset('living_thing.n.01'),
 Synset('whole.n.02'),
 Synset('object.n.01'),
 Synset('physical_entity.n.01'),
 Synset('entity.n.01')]
```

Discrete symbols to computer...



Discrete symbols to computer...



Before that ... in a nutshell



Raw Text



Tokenization

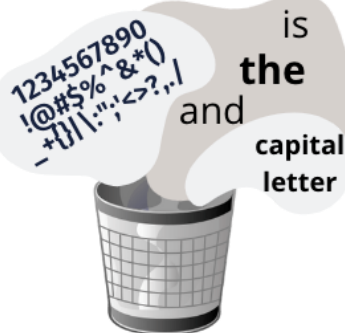
"This is tokenizing!"



['This', 'is',
'tokenizing', '!']



Text Cleaning



Normalization

Stemming

run^{ning} → run
ran → run

Lemmatization

leaves → leaf



Part-of-Speech (POS) Tagging

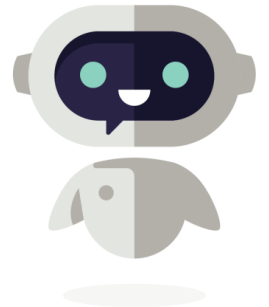
'This' → DT
'is' → VB
'tokenizing' → VB



Pre-processed Text



Encoding





One hot encoding...

1	0	0	0	0	0	0	0	0	0	study
0	1	0	0	0	0	0	0	0	0	machine
0	0	1	0	0	0	0	0	0	0	learn
										...

#-words in vocabulary



The vector dimension

Limitation : do not capture relations between words, no context

Bag of Words



Given quick brown fox jump lazy dog

lazy brown cat nap sunny windowsill

BoW

Text	Bag of Words Vector									
	quick	brown	fox	jump	lazy	dog	cat	nap	sunny	windowsill
Text 1	1	1	1	1	1	1	0	0	0	0
Text 2	0	1	0	0	1	0	1	1	1	1

Limitation : ignore word order

Term frequency Inverse Document Frequency



$$\text{Tf-Idf} = \text{Tf} \times \text{Idf}$$

$$= \left(\frac{f_d(t)}{\sum_{t \in d} f_d(t)} \right) \times \left(\log \left(\frac{|D|}{|\{d \in D : t \in D\}|} \right) \right)$$

$f_d(t)$: the number of occurrence word t in document d

$|D|$: the cardinality of set of documents D

$|\{d \in D : t \in D\}|$: the number of occurrence word t in D



Tf-Idf (simple form)

$$\text{Tf-Idf} = \text{Tf} \times \text{Idf}$$

$$= \left(\frac{f_d(t)}{\sum_{t \in d} f_d(t)} \right) \times \left(\log \left(\frac{N}{\text{DF}} \right) \right)$$

$f_d(t)$: the number of occurrence word t in document d

N : the number of documents D

DF : the number of occurrence word t in D

Tf-Idf



quick brown fox jump lazy dog

lazy brown cat nap sunny windowsill

- $TF(\text{brown, document 1}) = 1/6 = 0.1667$
- $DF(\text{brown}) = 2$ (it appears in both documents)
- $IDF(\text{brown}) = \log(2/2) = 0$
- $TF-IDF(\text{brown, document 1}) = 0.1667 * 0 = 0$

- $TF(\text{brown, document 2}) = 1/6 = 0.1667$
- $DF(\text{brown}) = 2$ (it appears in both documents)
- $IDF(\text{brown}) = \log(2/2) = 0$
- $TF-IDF(\text{brown, document 2}) = 0.2 * 0 = 0$

Text	TF-IDF									
	quick	brown	fox	jump	lazy	dog	cat	nap	sunny	windows ill
Text 1	0.0990	0	0.0990	0.0990	0	0.0990	0	0	0	0
Text 2	0	0	0	0	0	0	0.1155	0.1155	0.1155	0.1155

Let's back to discrete symbols...



1	0	0	0	0	0	0	0	0	0	study
0	0	1	0	0	0	0	0	0	0	learn

- The two vectors are **orthogonal** (in a mathematical sense)
- However, **no natural notion of similarity** for one hot vectors!
- Solution:
 - Integrate WordNet (~2010) ? → worse scenario (incompleteness, etc.)
 - **Encode similarity** → neural networks (deep learning)

Distributional semantics

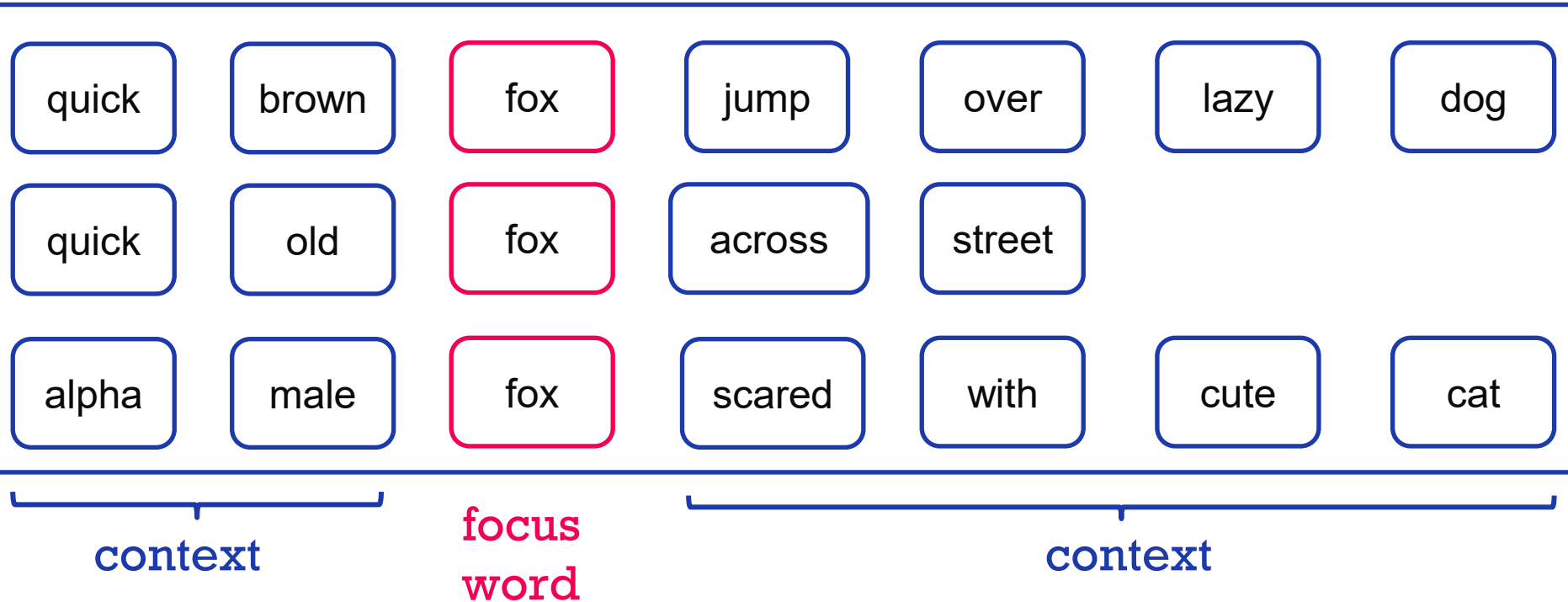


- Representing words from their context
- **Distributional semantics**: A word's meaning can be captured from the words that often occur close-by.
- J. R. Firth 1957: 11 ~ “*you shall know a word by the company it keeps*”
- The **context of a word** w is a set of words that locate nearby--within a fixed-size window when w appears.

Distributional semantics--example



- Use the many contexts of w to build up a representation of w



Word Vectors



- Construct a dense vector for each word
- **Word vectors** are also known **word embeddings** or **(neural) word representations** ~ **distributed representation**

$$\boxed{\text{fox}} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Word meaning visualization (high to low dimension)



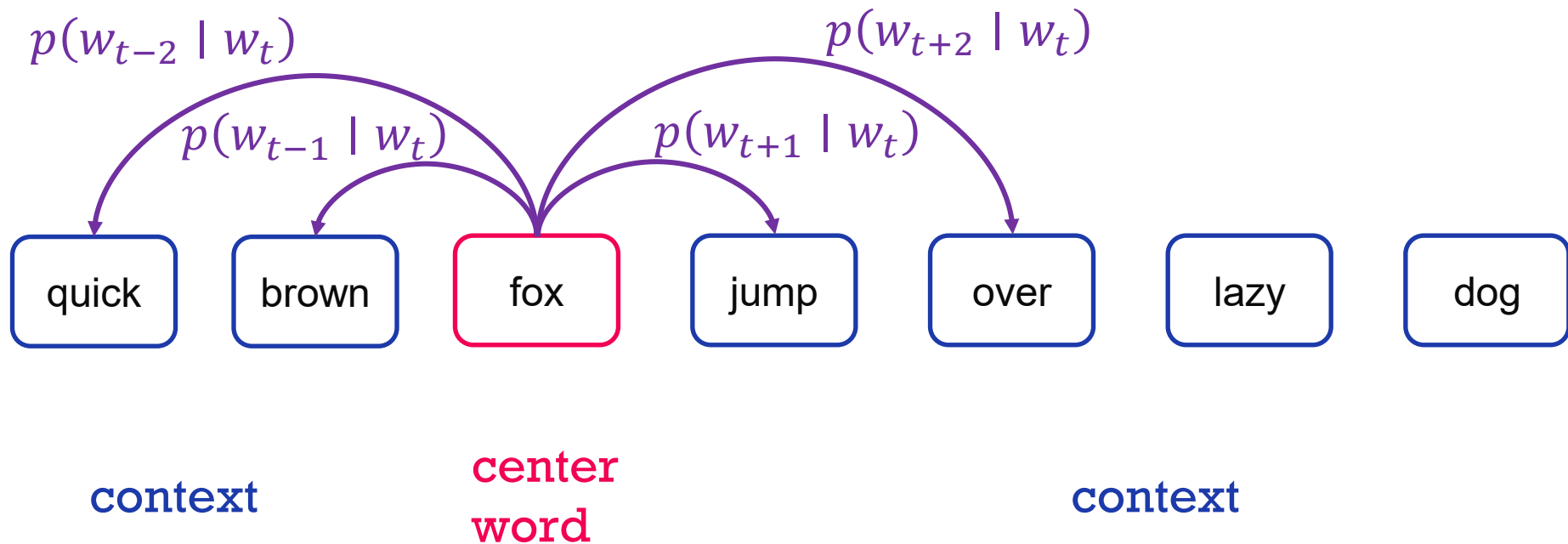


□ Idea :

- A **huge corpus** of text as the vocabulary
- **Each word** in vocabulary **represent to a vector**
- Each position word t in the text has a **center word** c and **context words** o .
- **Compute the similarity** of the word vectors for c and o from **the probability of o given c or vice versa.**
- **Maximize** the probability by **adjusting** the word vectors

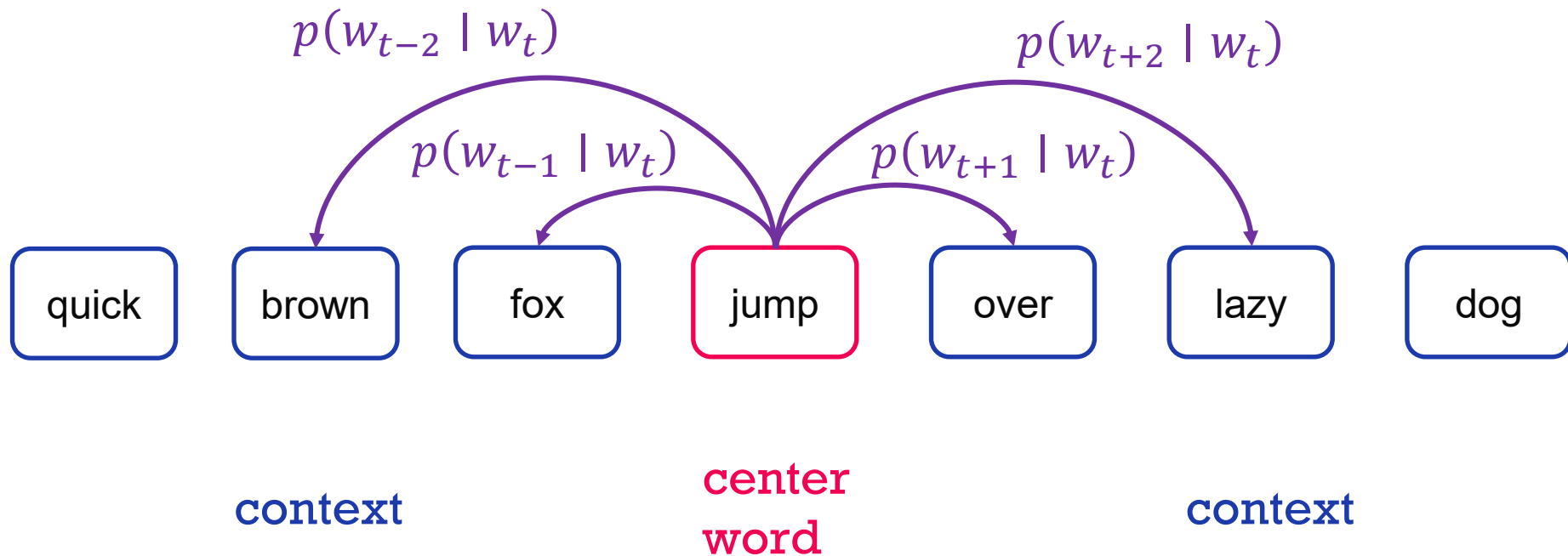


- Given window = 2





- Given window = 2





Maximize the probability

- Focus on each position word $t = 1, 2, \dots, T$
- **Predict context words** O within a fixed-window m size given a center word $c_j \rightarrow$ **Likelihood** $L(O | c)$
- Assume the model parameter θ

$$\max_{\theta} L_{\theta}(O | c) = \max_{\theta} \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} p_{\theta}(w_{t+j}, w_t)$$



Minimize the objective function

- Optimize all model parameters θ
- **Objective function** (a **loss** or **cost function**) : **average negative log-likelihood**

$$\min_{\theta} J_{\theta}(O | c) = \min_{\theta} -\frac{1}{T} \log L_{\theta}(O | c)$$

Note: $\log(a \cdot b) = \log a + \log b$

$$\min_{\theta} J_{\theta}(O | c) = \min_{\theta} -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log p_{\theta}(w_{t+j}, w_t)$$



Word2Vec steps:

- Minimize the objective function

$$\min_{\theta} J_{\theta}(O | c) = \min_{\theta} -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log p_{\theta}(w_{t+j}, w_t)$$

- But, **how to calculate** $p_{\theta}(w_{t+j}, w_t)$?
- Rely on two vectors per word: (i) **context words** u_o and (ii) **center word** v_c
- Use a Softmax function: $p(O | c) \approx \text{softmax}(O, c) = \frac{e(u_o^T v_c)}{\sum_{w \in d} e(u_w^T v_c)}$



A little bit of intuition from Softmax

Compute the similarity of O and c

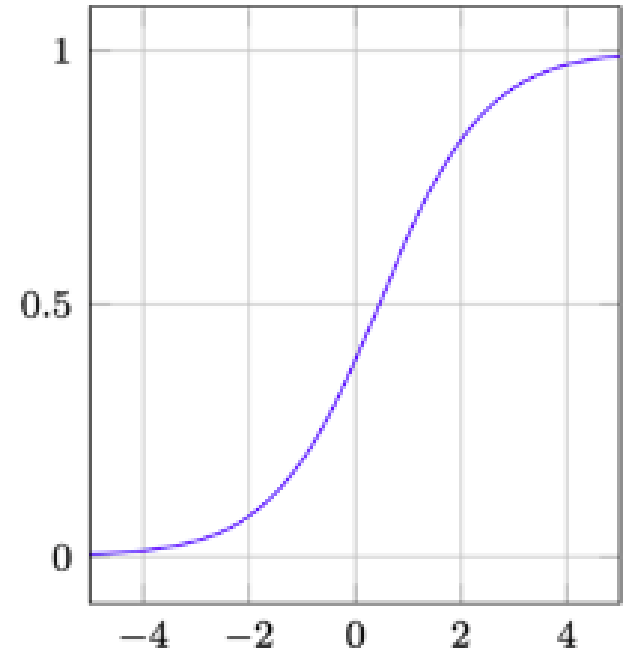
$$u^T v = u \cdot v = \sum_i u_i v_i$$

Ensure positive value

$$\text{Softmax}(O, c) = \sigma(O, c) = \frac{e^{u_O^T v_c}}{\sum_{w \in d} e^{(u_w^T v_c)}}$$

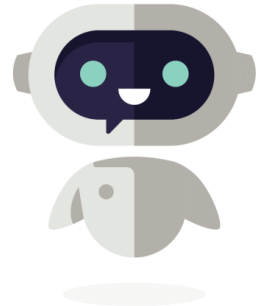
- Normalize over the entire vocabulary
- To obtain the probability

$$\sigma : \mathbb{R}^n \rightarrow (0,1)^n$$





Optimization Steps



Optimization steps



$$J_{\theta}(O \mid c) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log p_{\theta}(w_{t+j}, w_t)$$

$$\frac{\partial}{\partial v_c} J_{\theta}(O \mid c) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \frac{\partial}{\partial v_c} \left[\log \frac{e^{(u_o^T v_c)}}{\sum_{w \in d} e^{(u_w^T v_c)}} \right]$$

$$= -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \frac{\partial}{\partial v_c} \left[\log e^{(u_o^T v_c)} - \log \sum_{w \in d} e^{(u_w^T v_c)} \right]$$

Optimization steps



$$= -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \left[\frac{\partial}{\partial v_c} \left[\log e^{(u_o^T v_c)} \right] - \log \sum_{w \in d} e^{(u_w^T v_c)} \right]$$

recall

$$\frac{\partial}{\partial v_c} [u_o^T v_c] = u_o$$

$$\square \quad \frac{\partial}{\partial v_c} \left[\log e^{(u_o^T v_c)} \right] = \frac{\partial}{\partial v_c} [u_o^T v_c] = u_o$$

Optimization steps



$$\square \quad \frac{\partial}{\partial v_c} \left[\log \sum_{w \in d} e^{(u_w^T v_c)} \right]$$

Subs: $x = \sum_{w \in d} e^{(u_w^T v_c)} \rightarrow \frac{\partial x}{\partial v_c} = \frac{\partial}{\partial a} [\log \sum e^a] \frac{\partial a}{\partial v_c} \rightarrow \frac{\partial x}{\partial v_c} = \sum e^a \cdot u_{0'}$

$$\frac{\partial x}{\partial v_c} = u_{0'} \cdot \sum_{w' \in d} e^{(u_{w'}^T v_c)}$$

$$\frac{\partial}{\partial v_c} \left[\log \sum_{w \in d} e^{(u_w^T v_c)} \right] = \frac{\partial}{\partial v_c} [\log x] = \frac{\partial}{\partial x} [\log x] \frac{\partial x}{\partial v_c}$$

Optimization steps



$$\square \quad \frac{\partial}{\partial v_c} \left[\log \sum_{w \in d} e^{(u_w^T v_c)} \right] = \frac{\partial}{\partial x} [\log x] \frac{\partial x}{\partial v_c} = \frac{1}{x} \cdot u_{O'} \cdot \sum_{w' \in d} e^{(u_{w'}^T v_c)}$$

$$\frac{\partial}{\partial v_c} \left[\log \sum_{w \in d} e^{(u_w^T v_c)} \right] = \frac{1}{\sum_{w \in d} e^{(u_w^T v_c)}} \cdot u_{O'} \cdot \sum_{w' \in d} e^{(u_{w'}^T v_c)}$$

So,

$$\frac{\partial}{\partial v_c} [\log p(O | c)] = u_O - \frac{\sum_{w' \in d} e^{(u_{w'}^T v_c)}}{\sum_{w \in d} e^{(u_w^T v_c)}} \cdot u_{O'}$$



Optimization steps

$$\frac{\partial}{\partial v_c} [\log p(O | c)] = u_o - \frac{\sum_{w' \in d} e(u_{w'}^T v_c)}{\sum_{w \in d} e(u_w^T v_c)} \cdot u_{o'}$$

$$\frac{\partial}{\partial v_c} [\log p(O | c)] = u_o - \sum_{w' \in d} \frac{e(u_{w'}^T v_c)}{\sum_{w \in d} e(u_w^T v_c)} \cdot u_{o'}$$

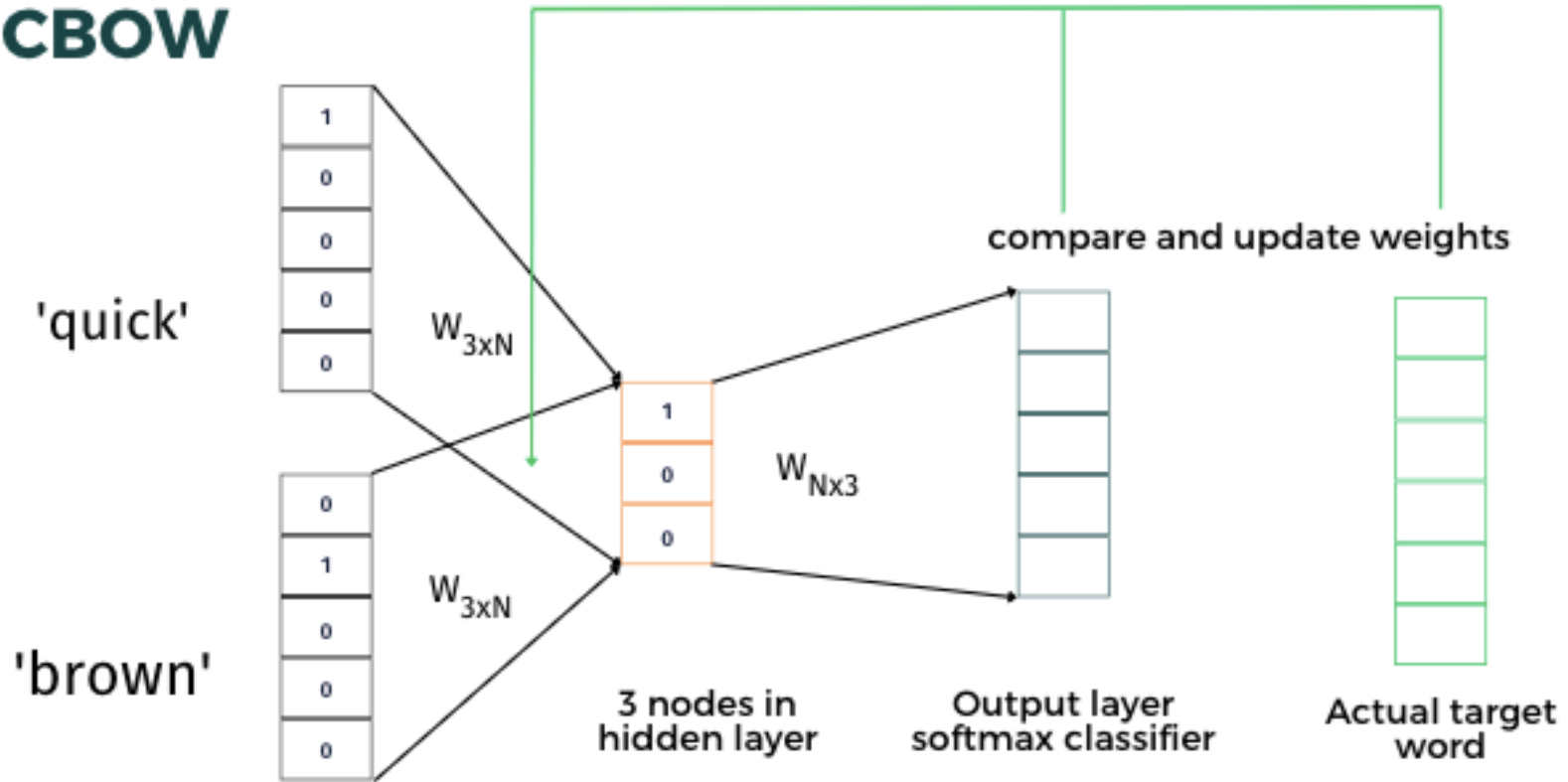
$$\frac{\partial}{\partial v_c} [\log p(O | c)] = u_o - \sum_{w' \in d} p(O | c) \cdot u_{o'}$$

observed expected



Continuous bag of words

CBOW





CBOW procedure

- Generate one hot vectors for every word.
- Given the size of window = m , the input one hot vectors or context is given by

$$(X^{(c-m)}, \dots, X^{(c-1)}, X^{(c+1)}, \dots, X^{(c+m)})$$

- Get **embedded word vector** for context.

$$V^{c+1} = W \times X^{c+1}$$

$$V^{c-1} = W \times X^{c-1}$$

$$\vdots$$

$$V^{c-m} = W \times X^{c-m}$$



CBOW procedure

- Calculate the context representation by taking the average of the embeddings of the **context words**.

$$\bar{V} = \frac{V^{(c-m)} + V^{(c-m+1)} + \dots + V^{(c+m-1)} + V^{(c+m)}}{2m}$$

- Generate the score vector for **target word**

$$z = \bar{V} \cdot W$$

- Calculate **probabilities** for each word using **Softmax Function**

$$\hat{z} = \text{softmax}(z)$$

- Obtain the final embedding representation by multiplying the probabilities to the respective embeddings.

Example



Vocabulary : ["quick", "brown", "fox", "jump"]

Context : ["quick", "brown"]

Target word : ["fox"]

One hot vector for each word :

- "quick": [1, 0, 0, 0]
- "brown": [0, 1, 0, 0]
- "fox": [0, 0, 1, 0]
- "jumps": [0, 0, 0, 1]

For simplicity, the **embedded word vector** are given in 2-dimensional vectors :

- "quick" = [0.5, 0.8]
- "brown" = [-0.1, 0.6]
- "fox" = [0.3, -0.6]
- "jump" = [-0.9, 0.5]



Example

- The context representation for **["quick", "brown"]** is given by

$$\begin{aligned}\bar{V} &= \frac{V^{(c-m)} + V^{(c-m+1)} + \dots + V^{(c+m-1)} + V^{(c+m)}}{2m} \\ &= \frac{V^{(c-1)} + V^{(c+1)}}{2(m)} = \frac{([0.5, 0.8] + [-0.1, 0.6])}{2(1)} = [0.2, 0.7]\end{aligned}$$

- Calculate the Score for the **Target Word ["fox"]**

$$\begin{aligned}z(\text{"fox"}) &= E(\text{fox}) = \hat{z}(\text{"fox"})W = [0.4108 \times 0.3, 0.4108 \times (-0.6)] \\ &= [0.12324, -0.24648] \\ &= 0.06 - 0.42 = -0.36\end{aligned}$$

- Apply Softmax Function to obtain **probability**

$$\begin{aligned}\hat{z}(\text{"fox"}) &= \text{softmax}(z) \\ &= \text{softmax}(-0.36) \approx 0.4108\end{aligned}$$

- The **final embeddings** for the target word will be

$$E(\text{fox}) = [0.4108 \times 0.3, 0.4108 \times (-0.6)] = [0.12324, -0.24648]$$



```
# linear model

def linear(m, theta):
    w = theta
    return m.dot(w)

# loss function

def log_softmax(x):
    e_x = np.exp(x - np.max(x))
    return np.log(e_x / e_x.sum())

def NLLLoss(logs, targets):
    out = logs[range(len(targets)), targets]
    return -out.sum()/len(out)

def log_softmax_crossentropy_with_logits(logits, target):

    out = np.zeros_like(logits)
    out[np.arange(len(logits)), target] = 1

    softmax = np.exp(logits) / np.exp(logits).sum(axis=-1, keepdims=True)

    return (- out + softmax) / logits.shape[0]
```



```
# forward function
```

```
def forward(context_idx, theta):  
    m = embeddings[context_idx].reshape(1, -1)  
    n = linear(m, theta)  
    o = log_softmax(n)
```

```
    return m, n, o
```

```
# backward function
```

```
def backward(preds, theta, target_idx):  
    m, n, o = preds  
  
    dlog = log_softmax_crossentropy_with_logits(n, target_idx)  
    dw = m.T.dot(dlog)
```

```
    return dw
```

```
# optimize function
```

```
def optimize(theta, grad, lr=0.03):  
    theta -= grad * lr  
    return theta
```



```
# data
sentences = """We are about to study the idea of a computational process.
Computational processes are abstract beings that inhabit computers.
As they evolve, processes manipulate other abstract things called data.
The evolution of a process is directed by a pattern of rules
called a program. People create programs to direct processes. In effect,
we conjure the spirits of the computer with our spells."""
```

```
# clean data
```

```
import re
```

```
# remove special characters
```

```
sentences = re.sub('[^A-Za-z0-9]+', ' ', sentences)
```

```
# remove 1 letter words
```

```
sentences = re.sub(r'(?::^| )\w(?:$| )', ' ', sentences).strip()
```

```
# lower all characters
```

```
sentences = sentences.lower()
```

```
words = sentences.split()
```

```
vocab = set(words)
```



```
# initialization

vocab_size = len(vocab)
embed_dim = 10
context_size = 2

# dictionary

word_to_ix = {word: i for i, word in enumerate(vocab)}
ix_to_word = {i: word for i, word in enumerate(vocab)}

# data bags

data = []
for i in range(2, len(words) - 2):
    context = [words[i - 2], words[i - 1], words[i + 1], words[i + 2]]
    target = words[i]
    data.append((context, target))
print(data[:5])

[(['we', 'are', 'to', 'study'], 'about'), (['are', 'about', 'study', 'the'],

```

```
# embedding

import numpy as np

embeddings = np.random.random_sample((vocab_size, embed_dim))
```



```
# training step
```

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('ggplot')

theta = np.random.uniform(-1, 1, (2 * context_size * embed_dim, vocab_size))

epoch_losses = {}

for epoch in range(80):

    losses = []

    for context, target in data:
        context_idxs = np.array([word_to_ix[w] for w in context])
        preds = forward(context_idxs, theta)

        target_idxs = np.array([word_to_ix[target]])
        loss = NLLLoss(preds[-1], target_idxs)

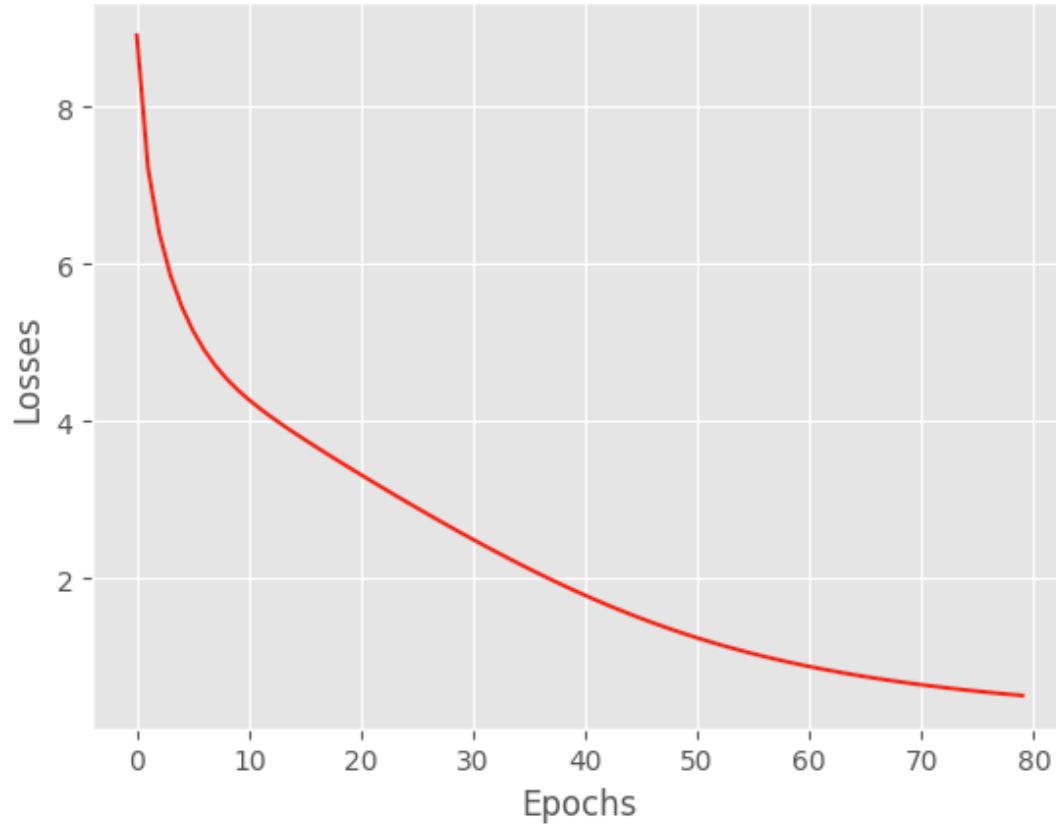
        losses.append(loss)

    grad = backward(preds, theta, target_idxs)
    theta = optimize(theta, grad, lr=0.03)

epoch_losses[epoch] = losses
```



Epoch/Losses





```
# prediction step

def predict(words):
    context_idx = np.array([word_to_ix[w] for w in words])
    preds = forward(context_idx, theta)
    word = ix_to_word[np.argmax(preds[-1])]

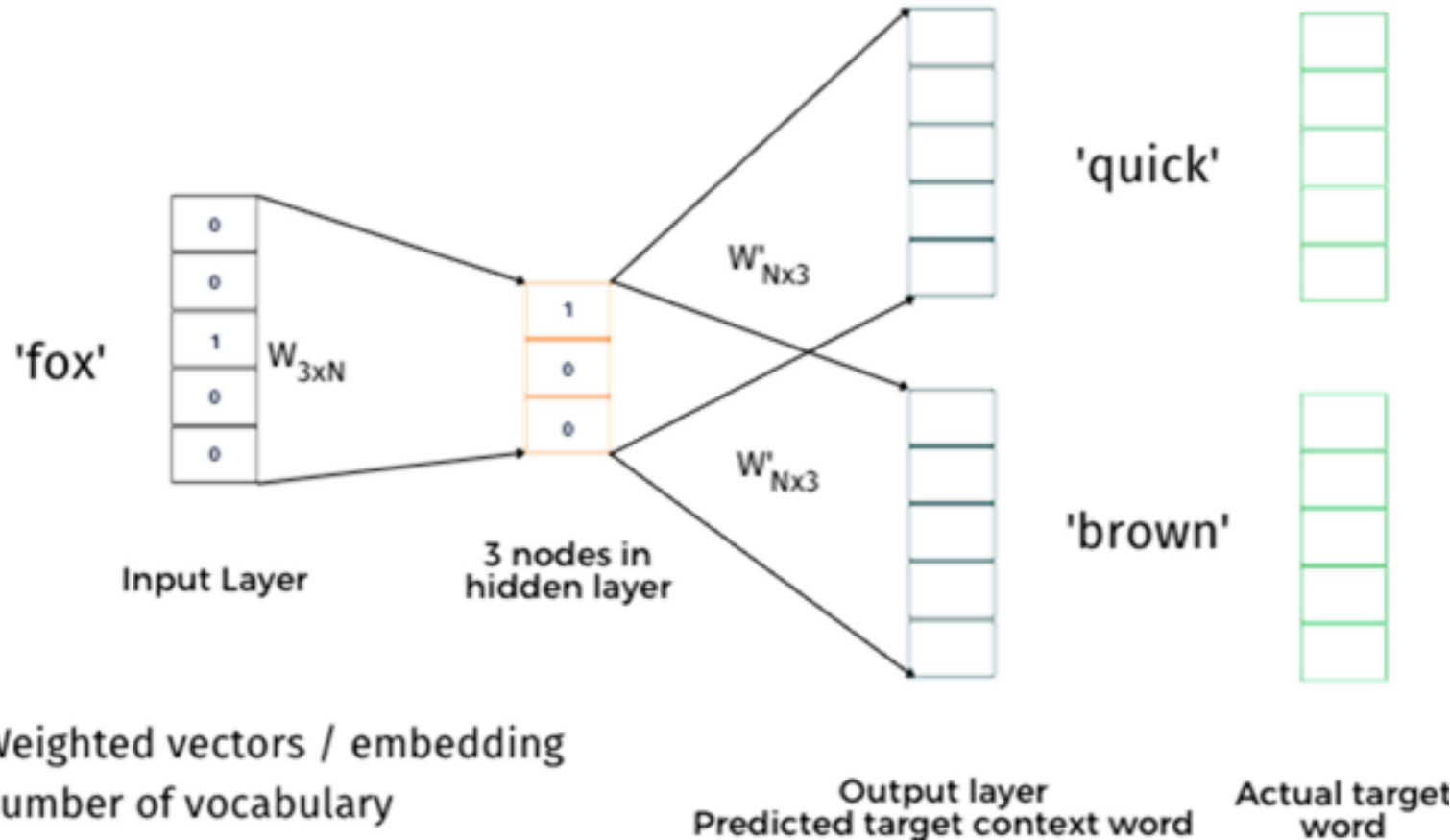
    return word, preds

w, p = predict(['we', 'are', 'to', 'study'])
print(w)

embbedNew = embeddings[0]*np.max(p[-1])
print(embbedNew)

about
[-0.31950797 -0.48310868 -0.01024366 -0.08850109 -0.16021292 -0.26937287
 -0.2289264  -0.08199722 -0.13122945 -0.46155476]
```

Skip Gram





Skip Gram procedure

- Initialize word embeddings matrix that includes every word in the vocabulary (randomly or using pre-trained embeddings).

$$W = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_n \end{bmatrix}, n: \text{number of vocabulary}$$

- The input layer represents the **target word**, and the output layer represents the **context word predictions**.
- Pass the target word's embedding (input) through the neural network and calculate scores for context words

$$z = v_w^T \cdot W$$

- Apply the Softmax Function to obtain the probabilities for each context word being a context word given the target word

$$\hat{z} = \text{softmax}(z)$$



example

Vocabulary : ["quick", "brown", "fox", "jump"]

Target word : ["fox"]

- Word embedding matrix :

$$W = \begin{bmatrix} [0.5, 0.8], \\ [-0.1, 0.6], \\ [0.3, -0.6], \\ [-0.9, 0.5] \end{bmatrix}$$

Word embedding for the **target word** "fox" is given by $v_{\text{fox}} = [0.3, -0.6]$

- For **each context word**, calculate the raw score

$$z_{\text{quick}} = v_{\text{fox}}^T v_{\text{quick}} = [0.3, -0.6]^T [0.5, 0.8] = -0.33$$

$$z_{\text{brown}} = v_{\text{fox}}^T v_{\text{brown}} = [0.3, -0.6]^T [-0.1, 0.6] = -0.39$$

$$z_{\text{jump}} = v_{\text{fox}}^T v_{\text{jump}} = [0.3, -0.6]^T [-0.9, 0.5] = -0.57$$



example

- Calculate the **probabilities** of each context word being a context word given the target word "fox".

$$\hat{z}_{\text{quick}} = \text{softmax}(z_{\text{quick}}) = \text{softmax}(-0.33) \approx 0.2028$$

$$\hat{z}_{\text{brown}} = \text{softmax}(z_{\text{brown}}) = \text{softmax}(-0.39) \approx 0.1239$$

$$\hat{z}_{\text{jump}} = \text{softmax}(z_{\text{jump}}) = \text{softmax}(-0.39) \approx 0.5$$

- The **final embeddings** for the target word will be

$$\begin{aligned} E &= \hat{z}_{\text{quick}}v_{\text{quick}} + \hat{z}_{\text{brown}}v_{\text{brown}} + \hat{z}_{\text{jump}}v_{\text{jump}} \\ &= 0.2028[0.5, 0.8] + 0.1239[-0.1, 0.6] + 0.5[-0.9, 0.5] \\ &= [0.1014, 0.1613] + [-0.0124, 0.0741] + [-0.1411, 0.0406] \\ &= [-0.0512, 0.2760] \end{aligned}$$

Sentence embedding



- Sentence embedding is a numerical representation of a sentence that captures its meaning and context.
- Unlike word embeddings which represent individual words, sentence embeddings represent entire sentences.
- Nowadays, sentence embeddings can be obtained from pre-trained language model such as: BERT and GPT.

Discussion...



Codes:

- <https://intip.in/CSIENTUST2023synonym>
- <https://intip.in/CSIENTUST2023CBOW>

Q n A

